

Exploring the Canopy of Constraint Modeling Languages

Ricardo Soto^{1,2} and Laurent Granvilliers¹

¹ LINA, Université de Nantes, France

² Escuela de Ingeniería Informática

Pontificia Universidad Católica de Valparaíso, Chile

{ricardo.soto, laurent.granvilliers}@univ-nantes.fr

Abstract. This paper focuses on the design of constraint programming languages. The primary design of a constraint modeling language using an object oriented approach is presented. General characteristics like object composition and inheritance are combined with specific features related to constraint domains. Software engineering aspects like reuse, extensibility, and project management are tackled. A system architecture is proposed.

1 Introduction

The design of constraint programming languages is a long story from the early days of artificial intelligence and constraint logic programming to recent developments of search oriented engines. The main dimensions of the space of languages are constraint modeling, constraint solving, mapping from modeling to solving, and computer programming. Our goal is to design a general constraint modeling language.

The task is hard due to the ubiquity of constraints in many application areas such as operations research, molecular biology, business applications, and engineering design. Several aspects must be considered such as the semantics of applications, the expertise of users, and the relevance of separating modeling and solving capabilities. The design of a new language will be investigated in the following from an object oriented point of view. The language is not currently implemented. This paper is submitted to the Constraint Modeling and Reformulation workshop with the aim of discussing with CP researchers.

1.1 Modeling and programming

The following principle is well known: the user states the constraint model and the system solves it. The constraint language is available at the user level. The solving engine is internal. Solving strategies are automatically defined.

Constraint logic programming languages are built on this principle. Constraint domains, namely constraint languages and associated solving techniques, are embedded in a logic-based computer programming language called the host

language. The host language provides many facilities such as unification, nondeterminism, and list management. The operational semantics is well-defined [19]. Several constraint domains were implemented: rational trees in Prolog III [7], algebraic constraints in CLP($\mathcal{R}$) [20], combinatorial problems in CHIP [8], interval constraints in CLP(BNR) [25]. Constraint processing and a generalization of logic programming were integrated in the Galileo language for engineering design [5]. NP-SPEC combines integer constraints with a limited Prolog-like language with no function symbols [6].

The use of logic programming concepts like nondeterminism in an imperative programming style led to the design of Alma-0 [1]. Constraint modeling using standard control abstractions is combined with programming of choice points.

The Mozart programming system is a generalization of the aforementioned systems, supporting several programming paradigms such as logic programming and constraint programming. It is based on the Oz language [30] and efficient constraint solving techniques [29].

Constraint solving libraries were developed using object-oriented programming languages. ThingLab is a Smalltalk implementation of a visual environment for manipulating constrained geometric objects [4]. ILOG Solver is a C++ library mainly designed for solving combinatorial problems [28]. JACK is a Java interface for the constraint handling rules [14].

1.2 Modeling and solving

Several modeling languages have been designed in the mathematical programming area, for instance GAMS [3], AMPL [11], and Numerica [35]. Many subareas have been defined such as linear programming, quadratic programming, and dynamic programming. General models of optimization problems can be defined and processed using general purpose solving algorithms.

Combinatorial optimization seems more complex since there is no universal algorithm. ALICE was a pioneering language for constraint modeling and solving [23]. Efficient solving may require specific search strategies [32]. Many languages were designed for global search, local search and hybrid search in the constraint programming area [33, 34, 22]. Such approaches were also experimented in mathematical programming, introducing search primitives into algebraic modeling languages [2].

The separation of modeling and solving aspects is a recent concern. A three-layered system architecture is proposed in general, including a modeling language, a solving engine, and a middle tool mapping constraint models to solvers. The Essence language [13] is combined with the Minion solver. The Zinc modeling language [27] is projected onto many solvers using the Cadmium language. The ESRA relational modeling language [10] is translated towards many other languages. These modeling languages include first-order constructs, data structures, and discrete constraint domains.

Alloy is a modeling language allowing to express complex structures using full first-order logic [18]. Its notation is inspired from the formal specification language Z. Systems can be described by collections of constraints that are solved

by means of SAT solvers. Constraint Lingo is a language devoted to tabular constraint satisfaction problems [9]. $\mathcal{F}$ is a language based on function modeling [17]. It was specifically designed for constraint problems expressed as functions between two sets such as map coloring.

1.3 Modeling with constrained objects

Objects can be used to encapsulate problem structures. Constrained objects are objects subject to constraints on their attributes. In this approach, the state of objects is controlled by constraint solving.

ThingLab [4], successor of Sketchpad [31], was designed for interactive graphical simulation using constrained geometric objects. In ThingLab, the constraint domain is fixed and the class system can be extended, using composition and inheritance. The state of the system through interactive operations is controlled by geometric constraint solving. This approach was further developed in the Kaleidoscope language [12].

Gianna is a visual environment for modeling constraint-based problems with objects [26]. Object constraint models are transformed into equivalent constraint satisfaction problems. Similar ideas were proposed for automatic configuration applications [16]. The COB language merges constrained objects, first-order formulas, and predicates [21]. Constraints may be atomic from several domains, conditional, quantified, or soft. Constraint solving requirements are identified. The spectrum is very large, including differential equations.

OCL is a constraint language on UML diagrams used to define invariants on objects and conditions on operations [36]. This language is very specific and not designed for modeling constrained problems.

1.4 Contribution

Our working hypothesis is to define a universal constraint modeling language independent from solving concerns. Object oriented modeling is used to tackle problem structures and project management aspects of large scale applications. First-order logic allows one to define general constraint-based formulas. Primitives are given to introduce user-defined constraint domains. Classical data structures and control abstractions are present. Direct applications are found in the design and configuration area.

The declarative semantics of the language amounts to the equivalence of every model to some first-order formula over the given constraint domains.

This work can be seen as a combination of Kaleidoscope or COB for constrained objects and Zinc for other logic and programming constructs. The ability of extending the system of constraint domains is an important feature supporting the generality of the language. We plan to make our language open source for the research community.

The outline of this paper is as follows. The main characteristics of the language are presented in Section 2. Constraint modeling examples are described in

Section 3. Section 4 is devoted to the system architecture. Concluding remarks and directions for further research follow.

2 Main Features

The main features of the language are motivated in this section, namely the core of the language, important characteristics from the user point of view, and the system architecture for parsing.

2.1 Core

High-Level Syntax. The object part of the language is an abstraction of the Java programming style. Classical constructs like classes, basic data types, data structures, conditional statements, and iteration statements are defined using a simple notation. Assignment is not implemented except for object construction.

Object modeling. This style is interesting for several reasons. It provides the required expressiveness for representing problems with structures. It is simple enough for use by non specialists of computer science.

Constraint modeling. The space of constraints can be extended. New constraint domains can be given by regular expressions and grammars. Every model is equivalent to a first-order formula. Specific constructs are also supported such as activity constraints, compatibility constraints, and global constraints.

2.2 User level

Hierarchy. The hierarchical class diagram is a good basis for managing large scale constraint problems. Classes, data files, and extension files can be gathered to build components. These components can be stored in databases.

Reuse. Abstraction, data encapsulation, composition, inheritance, and polymorphism are main features of the language supporting reuse and extension of existing components.

Extensibility. The easy specification of user-defined constraint domains makes the language general. The built-in first-order constructs allow the definition of general formulas over the constraint domains.

2.3 System level

Separation of data from model. The goal is to build models from libraries, namely standard modeling components or data files.

Solver independent modeling. A general language cannot be associated to specific solving techniques. In a first approach, we are interested only in the modeling language.

Parsing. The parser of the language is decomposed into a parser for the base language and parsers for constraint domains. Parsers for user-defined constraint domains are generated automatically. This approach allows one to easily maintain the parsing engine, not to recompile the base language. It requires specific rules to handle ambiguity and typing problems.

3 Tour of the Language

The language is illustrated by means of an academic problem from the configuration and design area. We consider the task of configuring a car engine using a hierarchical and compositional approach (see Figure 1). The engine at the first level is built from a crankcase, a cylinder system, a block and a cylinder head at the second level. The cylinder system is a subsystem made of a valve system, a piston system and either an injection or a carburetor. Both valve and piston systems have their own composition rules. This problem allows one to focus on problem structures, data encapsulation, and other object mechanisms.

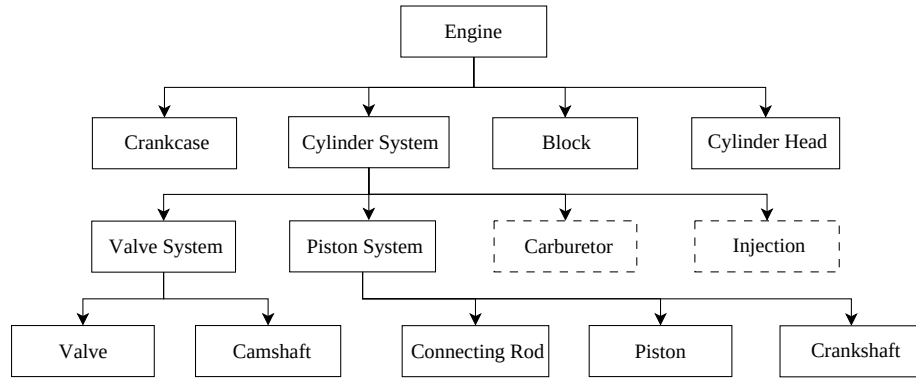


Fig. 1. Hierarchical car engine structure (dashed lines represent optional elements).

We then define a model represented by a set of classes. Every class is defined by attributes and constraints. The attributes may represent constant or variable quantities, from simple types or more complex object types. The constraints must be satisfied by every instance of the given class.

3.1 Main class

The Engine class is at the top of the hierarchy. The attributes `cCase`, `cSyst`, `block` and `cHead` represent the subsystems of the engine. Those four objects are

instances of classes to be declared. The last attribute `cm3` from the built-in `real` type defines the volume of the engine.

```
class Engine {
    CrankCase cCase;
    CylSystem cSyst;
    Block     block;
    CylHead   cHead;
    real      cm3;
}
```

The other classes and attributes are described in the same way, for instance the `CrankCase` class below.

```
class CrankCase {
    string type;
    real   oilVesselVolume;
    real   bombePower;
}
```

Data is expressed declaring objects with its values in the data file. Constructors are not needed. Attributes not present in the data file are considered variables. In the example `type` is considered variable, `oilVesselVolume` and `bombePower` are considered constants.

```
cCase {
    oilVesselVolume := 7.5;
    bombePower      := 105.8;
}
```

3.2 Activity constraints

The class `CylSystem` has a more complex declaration. The first `type` constrained attribute represents the cylinders configuration of the form *ab* where *a* is the number of cylinders and *b* is the structure (in line or V). The cylinder system also has four subsystems denoted by `carb`, `inj`, `vSyst`, and `pSyst`. This problem can be seen as an activity constraint in the dynamic CSP framework [24]. This activity constraint represents the fact that only 4L-engines with 2 valves per cylinder have a carburetor instead of an injection. Others kinds of engines must have an injection.

```
class CylSystem {
    string      type in {4L,6L,6V,8V};
    ValveSystem vSyst;
    PistonSystem pSyst;
    constraint injOrCarb {
        if ((type=4L) AND (vSyst.valvePerCyl=2))
            Injection inj;
        else

```

```

        Carburetor carb;
    }
}

```

Block and CylHead are simple classes without subsystems.

```

class Block {
    string type;
    real left;
    real right;
}

class CylHead {
    string type;
    real height;
    real volume;
}

```

The valve system is a subsystem of the cylinder system, having a constrained attribute `valvePerCyl` and two objects `valve` and `caShaft`. Finally four additional attributes address some features of the valve system.

```

class ValveSystem {
    int valvePerCyl in {2,4};
    Valve valve;
    CamShaft caShaft;
    real rockArmHeight;
    real rockArmSpringHeight;
    real springDiameter;
    string ThrustNutType;
}

```

3.3 Arithmetic constraints

The second subsystem of the cylinder system is the piston system. This subsystem is composed of a `ConRod` object, a `Piston` object, a `CrankShaft` object and a attribute called `crankRPM`. The piston system is subject to a numeric constraint between the `bottomDiameter` attribute of `cRod` and the `diameter` attribute of `pist`.

```

class PistonSystem {
    ConRod cRod;
    Piston piston;
    CrankShaft crShaft;
    real crankRPM;
    constraint diameter {
        cRod.bottomDiameter < 0.5*pist.diameter;
    }
}

```

3.4 Compatibility constraints

The subsystem injection is composed of three constrained attributes called `type`, `admValve`, and `pressure`. The injection class has also a compatibility constraint [15] between its components.

```

class Injection {
  string type in {direct, indirect};
  string admValve in {small,medium,large};
  real pressure in [100,140];
  compatibility constraint
    C(type,admValve,pressure,parent.type) {
      (direct, small, [100,110], 4L);
      (indirect, medium, [110,120], 6L);
      (indirect, medium, [110,120], 6V);
      (indirect, large, [130,140], 8V);
    }
}

```

3.5 Inheritance

Remaining elements of the engine are avoided because they can be modeled with the elements already presented in previous subsections.

Consider now the task of configuring a turbo engine. This kind of engine presents similar features to a normal engine, hence we can define this new class as a subclass of the class engine. Using the reserved word **extends** a subclass inherits all attributes and constraints defined in its superclass.

```

class TurboEngine extends Engine {
  turboCharger turbo;
}

class TurboCharger {
  int boost in {5,6,7,8};
  string turbineWheel in {small, medium, large};
  string compressorWheel in {small, medium, large};
}

```

3.6 Polymorphism

Consider a new subclass called **Piston2** inheriting both attributes and constraints from its **Piston** superclass. The inherited constraint can be overridden as follows.

```

class Piston {
  string type;
  real height;
  real volume;
  real diameter;
  real velocity;
  constraint dimension {
    height < 33.5 AND
    diameter < 8.7;
  }
}

class Piston2 extends Piston {
  constraint dimension {
    height < 39;
  }
}

```


3.7 Model import

Consider the task of configuring a car. Every subsystem may be stored in different files to be imported in the main file. This mechanism provides a clearer and more understandable model.

```
import "Engine.mod";
import "ElectSystem.mod";
...

class Car {
    Engine      eng;
    ElectSystem elSyst;
    ExhaustSystem exSyst;
    SuspSystem  suSyst;
    DriveTrain  drTrain;
    Chassis     chass;
    ...
}
```

3.8 User-defined constraint domains

A main aspect is the capability of extending the constraint language. To this end a triple has to be defined, a set of constants, a set of function symbols, and a set of relation symbols. This is just the basis of a first-order language. Consider a restricted data type called **size** with values {small,medium,large}. The declaration is as follows, using regular expressions.

```
Constant {
    size: ('small'|'medium'|'large');
}
```

The use of a new type requires an import statement, as follows. Every attribute of type **size** is a variable lying in the set of size values.

```
import "size.ext";

class Carburetor {
    size    admValve;
    size    admGarg;
    string  injectorType;
}
```

Consider the more complex definition of constraints based on rational numbers. Every symbol is characterized by a priority, an arity, and a notation (infix, prefix, postfix).

```
Constant {
    rat: ('0'..'9')+ '/' ('1'..'9')('0'..'9')+;
```

```

}

Function {
  + (2,i) : rat x rat -> rat;
  - (2,i) : rat x rat -> rat;
  - (1,p) : rat          -> rat;
  * (1,i) : rat x rat -> rat;
  / (1,i) : rat x rat -> rat;
}

Relation {
  > (i) : rat x rat;
  >= (i) : rat x rat;
  < (i) : rat x rat;
  <= (i) : rat x rat;
  = (i) : rat x rat;
  <> (i) : rat x rat;
}

```

3.9 Global constraints

Global constraints can be declared as extensions of the constraint language or user-defined constraints. Consider the alldifferent constraint. The language extension is as follows, defining a prefix symbol on integers of any arity greater than one.

```

Relation {
  alldifferent (p) : int +;
}

```

An equivalent user-defined constraint is given below.

```

constraint alldifferent (int list l) {
  for i:=1 to l.size-1 do {
    for j:=i to l.size-1 do
      l[i] <> l[j+1];
  }
}

```

4 System Architecture

In the long term we propose to build a tree-layered system including modeling, solving, and mapping from modeling to solving, as in other developments like G12 (see Figure 2). Models are composed of multiple submodels, extension files, and data files. The parser output is a syntactic tree of the constraint-based model encoded using standard XML notations.

The parser is implemented as one parser for the base language and a set of parsers, each one for a user-defined constraint domain from an extension file

(see Figure 3). The idea is to generate automatically those parsers. The main problem is that ambiguity may happen in the whole language. A control layer is introduced to manage this problem. Several tests are demanded:

- The type of a constraint is determined at the lexical analysis stage. Once the type is known, for instance from variable types, the entry in the grammar is known and the corresponding parsing tool is called. The lexer follows a look ahead strategy, storing many tokens before the grammar analysis.
- Reserved keywords must be unique. A table of symbols is managed at the control layer and it is used to determine the correct token.
- Constraint symbols can be polymorphic due to the typing operation.

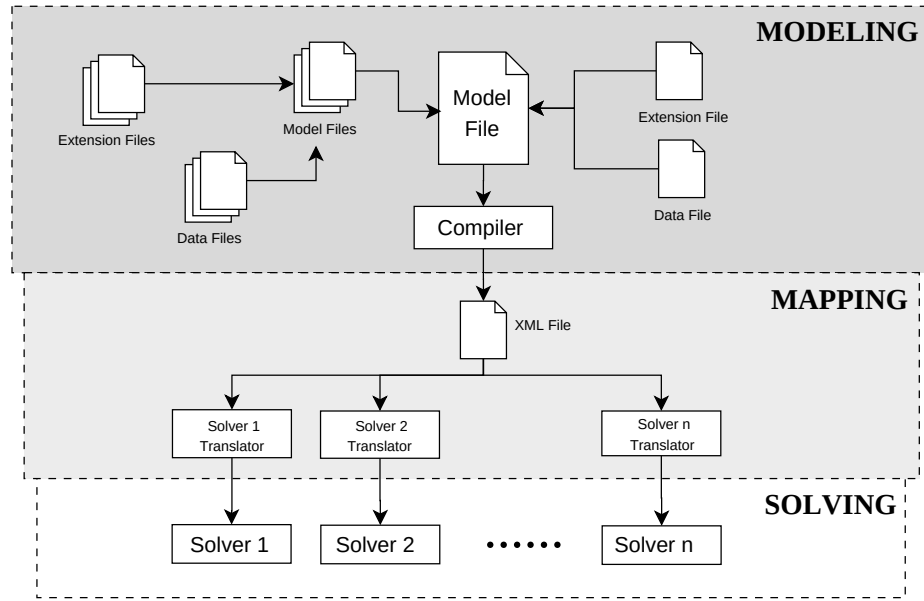


Fig. 2. Three-layered system architecture.

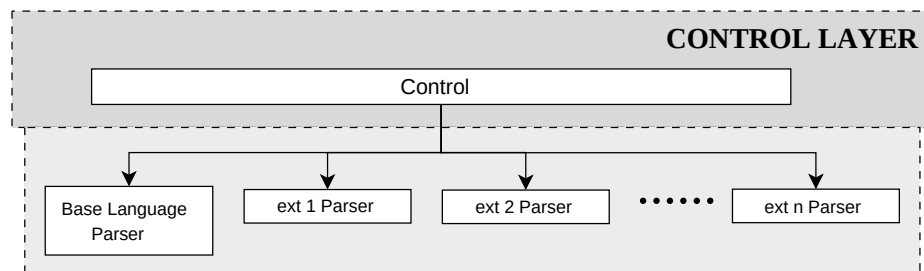


Fig. 3. The parser structure.

5 Future Research

This work is at a preliminary stage. Many points have to be discussed or refined.

- The object oriented style seems important to us since we believe that non CP experts may be able to easily state constraint models. Graphical interfaces may be useful as in the UML world. The discussion with end users is still to be done.
- The extension of the constraint language is a main feature. Many aspects have to be studied such as ambiguity and typing problems, casting operations (e.g., int to real), heterogeneous constraints.
- Project management facilities have to be developed in the spirit of making databases of constraint-based components.
- Further work on the design of elementary constraints such as data structures, and necessary built-in components is demanded.
- The implementation is under study using parser generation tools like ANTLR.
- We plan to develop the whole modeling and solving chain for emphasizing specific features such as structure-based solving.

Acknowledgments

We are grateful to the support of this research from the “Pontificia Universidad Católica de Valparaíso” under the grant “Beca de Estudios Básica”, and to Carlos Castro and Éric Monfroy who have discussed with us on this topic.

References

1. K.R. Apt, J. Brunekreef, V. Partington, and A. Schaerf. Alma-0: An imperative language that supports declarative programming. *ACM TOPLAS*, 20(5):1014–1066, 1998.
2. J.J. Bisschop and R. Fourer. New Constructs for the Description of Combinatorial Optimization Problems in Algebraic Modeling Languages. *Combinatorial Optimization and Applications*, 6:83–116, 1996.
3. J.J. Bisschop and A. Meeraus. On the Development of a General Algebraic Modeling System in a Strategic Planning Environment. *Mathematical Programming Studies*, 20:1–29, 1982.
4. A.H. Borning. The Programming Languages Aspects of ThingLab, a Constraint-Oriented Simulation Laboratory. *ACM TOPLAS*, 3(4):353–387, 1981.
5. J. Bowen, P. O’Grady, and L. Smith. A Constraint Programming Language for Life-Cycle Engineering. *AI in Engineering*, 5(4):206–220, 1990.
6. M. Cadoli, G. Ianni, L. Palopoli, A. Schaerf, and D. Vasile. An Executable Specification Language for Solving All the Problems in NP. *Computer Languages*, 26(2–4):165–195, 2000.
7. A. Colmerauer. An Introduction to Prolog III. *Communications of the ACM*, 33(7):69–90, 1990.
8. M. Dincbas, H. Simonis, and P. Van Hentenryck. Solving Large Combinatorial Problems in Logic Programming. *JLP*, 8:75–93, 1990.

9. R.A. Finkel, V.W. Marek, and M. Truszczyński. Constraint Lingo: Towards High-level Constraint Programming. *Software Practice and Experience*, 34(15):1481–1504, 2004.
10. P. Flener, J. Pearson, and M. Agren. Introducing ESRA, a Relational Language for Modelling Combinatorial Problems. In *LOPSTR*, Verona, Italy, 2004.
11. R. Fourer, D.M. Gay, and B.W. Kernighan. A Modeling Language for Mathematical Programming. *Management Science*, 36:519–554, 1990.
12. B.N. Freeman-Benson and A. Borning. Integrating Constraints with an Object-Oriented Language. In *ECOOP*, Utrecht, The Netherlands, 1992.
13. A.M. Frisch, C. Jefferson, B. Martinez-Hernandez, and I. Miguel. The Rules of Constraint Modelling. In *IJCAI*, Edinburgh, Scotland, 2005.
14. T. Frühwirth. Theory and Practice of Constraint Handling Rules. *JLP*, 37(1–3):95–138, 1998.
15. E. Gelle and B. Faltings. Solving mixed and conditional constraint satisfaction problems. *Constraints*, 8(2):107–141, 2003.
16. T. Hinrich, N. Love, C. Petrie, L. Ramshaw, A. Sahai, and S. Singhal. Using Object-Oriented Constraint Satisfaction for Automated Configuration Generation. In *DSOM*, Davis, USA, 2004.
17. B. Hnich. *Function Variables for Constraint Programming*. PhD thesis, Uppsala University, 2003.
18. D. Jackson. Automating First-Order Relational Logic. In *SIGSOFT FSE*, San Diego, USA, 2000.
19. J. Jaffar and J.L. Lassez. Constraint Logic Programming. In *ACM POPL*, Munich, Germany, 1987.
20. J. Jaffar, S. Michaylov, P. Stuckey, and R. Yap. The CLP($\mathbb{R}$) Language and System. *ACM TOPLAS*, 14(3):339–395, 1992.
21. B. Jayaraman and P.Y. Tamba. Constrained Objects for Modeling Complex Structures. In *OOPSLA*, Minneapolis, USA, 2000.
22. F. Laburthe and Y. Caseau. SaLSA: A Language for Search Algorithms. In *CP*, Pisa, Italy, 1998.
23. J.L. Laurière. A Language and a Program for Stating and Solving Combinatorial Problems. *Artificial Intelligence*, 10(1):29–127, 1978.
24. S. Mittal and B. Falkenhainer. Dynamic constraint satisfaction problems. In *Proc. of the Eight National Conference on Artificial Intelligence (AAAI-90)*, pages 25–32. The MIT Press, 1990.
25. W. J. Older and A. Vellino. Extending Prolog with Constraint Arithmetic on Real Intervals. In *IEEE ECE*, New York, 1990.
26. M. Paltrinieri. On the Design of Constraint Satisfaction Problems. In *PPCP*, Orcas Island, USA, 1994.
27. G12 Project. Initial Specifications of Zinc. Technical report, National ICT Australia, 2006.
28. J.F. Puget. A C++ implementation of CLP. In *SCIS*, Singapore, 1994.
29. C. Schulte. *Programming Constraint Services*. Springer Verlag, 2002.
30. G. Smolka. The oz programming model. In *Computer Science Today*, volume 1000 of *LNCS*, pages 324–343. 1995.
31. I.E. Sutherland. Sketchpad: A Man-Machine Graphical Communication System. In *IFIPS*, Detroit, USA, 1963.
32. P. Van Hentenryck. A Logic Language for Combinatorial Optimization. *Annals of Operations Research*, 21:247–273, 1989.
33. P. Van Hentenryck. *The OPL Optimization Programming Language*. The MIT Press, 1999.

- 34. P. Van Hentenryck and L. Michel. *Constraint-Based Local Search*. The MIT Press, 2005.
- 35. P. Van Hentenryck, L. Michel, and Y. Deville. *Numerica: a Modeling Language for Global Optimization*. MIT Press, 1997.
- 36. J.B. Warmer and A.G. Kleppe. *The Object Constraint Language*. Addison Wesley, 2003.